INSTALACIÓN Y CONFIGURACIÓN DE HERRAMIENTA DE VERSIONAMIENTO (LOCAL / WEB)

GA7-220501096-AA1-EV04

MICHAEL RONALD OLIVARES GIRALDO

SERVICIO NACIONAL DE APRENDIZAJE – SENA

REGIONAL DISTRITO CAPITAL - CENTRO METALMECÁNICO

FORMACIÓN TITULADA VIRTUAL Y A DISTANCIA

TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE

FICHA 3118306

FASE 3 EJECUCION

INSTRUCTOR: ING. LEONARDO MORENO COLLAZOS

MARZO DEL 2026

Evidencia de desempeño: **GA7-220501096-AA1-EV04 INSTALACIÓN Y CONFIGURACIÓN DE HERRAMIENTA DE VERSIONAMIENTO (LOCAL / WEB)**

Tomando como referencia el componente formativo “Integración continua”, realice la instalación y configuración de las herramientas de control de versionamiento tanto local como remoto.

Elementos para tener en cuenta en el documento:

- ✓ Se deben seguir las normas básicas de presentación de un documento escrito, es decir el documento debe tener como mínimo una portada, introducción, objetivo, paso a paso con pantallazos de la instalación de las herramientas de control de versionamiento tanto local como remota.

Lineamientos generales para la entrega de la evidencia:

- ✓ Productos para entregar: documento instalación
- ✓ Extensión: Libre.
- ✓ Formato: PDF.

Para hacer el envío de la evidencia remítase al área de la actividad correspondiente y acceda al espacio: **INSTALACIÓN Y CONFIGURACIÓN DE HERRAMIENTA DE VERSIONAMIENTO (LOCAL / WEB) GA7-220501096-AA1-EV04**

1. INTRODUCCIÓN

El presente documento constituye la evidencia GA7-220501096-AA1-EV04 - Instalación y Configuración de Herramienta de Versionamiento (Local / Web), en el marco de la Fase 3 de Ejecución del programa Tecnología en Análisis y Desarrollo de Software, SENA Ficha 3118306.

Tomando como referencia el componente formativo "Integración Continua" y los documentos previos de esta fase — GA7-220501096-AA1-EV01 (Plan de Trabajo), GA7-220501096-AA1-EV02 (Estándares de Codificación) y GA7-220501096-AA1-EV03 (Identificación de Herramientas de Versionamiento), el presente informe técnico registra de manera detallada el proceso de instalación y configuración de Git como herramienta de control de versiones local, y de GitHub como plataforma de versionamiento remoto, aplicados al desarrollo del Sistema de Diseño Integrado de Circuitos v3.0.

El control de versiones es un componente crítico del ciclo de vida del software, ya que permite registrar el historial completo de cambios, revertir estados anteriores del código, gestionar ramas de desarrollo paralelas y automatizar la integración y entrega continua (CI/CD) mediante flujos de trabajo en GitHub Actions.

El documento incluye el paso a paso detallado con capturas de pantalla de: la descarga e instalación de Git para Windows, la configuración inicial de identidad en el repositorio local, la inicialización del repositorio del proyecto, la creación de la cuenta en GitHub, la configuración de autenticación SSH, la creación del repositorio remoto, la vinculación entre el repositorio local y el remoto, y la verificación del primer push al servidor.

2. OBJETIVOS

2.1. Objetivo General

Instalar y configurar las herramientas de control de versionamiento Git (local) y GitHub (remoto) para el desarrollo del Sistema de Diseño Integrado de Circuitos v3.0, documentando cada paso del proceso mediante capturas de pantalla, en el marco de la Fase 3 de Ejecución del programa Tecnología en Análisis y Desarrollo de Software, SENA Ficha 3118306.

2.2. Objetivos Específicos

- Registrar el proceso de descarga e instalación de Git 2.44 en el sistema operativo Windows 11, documentando cada opción de configuración seleccionada durante el asistente de instalación.
- Documentar la configuración inicial de Git local mediante los comandos git config --global, estableciendo la identidad del desarrollador (nombre, correo y rama principal por defecto).
- Registrar la inicialización del repositorio local del Sistema de Diseño Integrado de Circuitos v3.0 mediante git init, incluyendo la estructura de directorios y el primer commit.
- Documentar la creación de la cuenta en GitHub y la configuración del repositorio remoto michael-olivares/sistema-diseno-circuitos-v3.
- Registrar la configuración del método de autenticación SSH entre el equipo local y GitHub, incluyendo la generación del par de claves Ed25519 y la adición de la clave pública al perfil de GitHub.
- Documentar la vinculación del repositorio local con el repositorio remoto mediante git remote add origin y la verificación del primer push a la rama main.

3. PRERREQUISITOS Y RECURSOS NECESARIOS

Antes de iniciar el proceso de instalación y configuración, se verificaron los requisitos del sistema. La siguiente tabla resume los prerrequisitos técnicos del entorno de desarrollo:

Componente	Requisito	Versión utilizada
Sistema Operativo	Windows 10 / 11 (64 bits)	Windows 11 Home 23H2
Procesador	x64 compatible	Intel Core i5 (11.ª gen)
RAM	4 GB mínimo	8 GB
Espacio en disco	500 MB libres	Disponible en C:\
Conexión a internet	Requerida para GitHub	Wi-Fi / Ethernet
Editor de código	Visual Studio Code (recomendado)	VS Code 1.87.2

Tabla 1. Requisitos del sistema para la instalación de herramientas de versionamiento.

4. INSTALACIÓN DE GIT PARA WINDOWS

4.1. Descarga del instalador de Git

El primer paso consiste en descargar el instalador oficial de Git para Windows desde el sitio web git-scm.com/download/win. La plataforma detecta automáticamente la arquitectura del sistema operativo y ofrece el instalador de 64 bits. La versión descargada es Git 2.44.0.

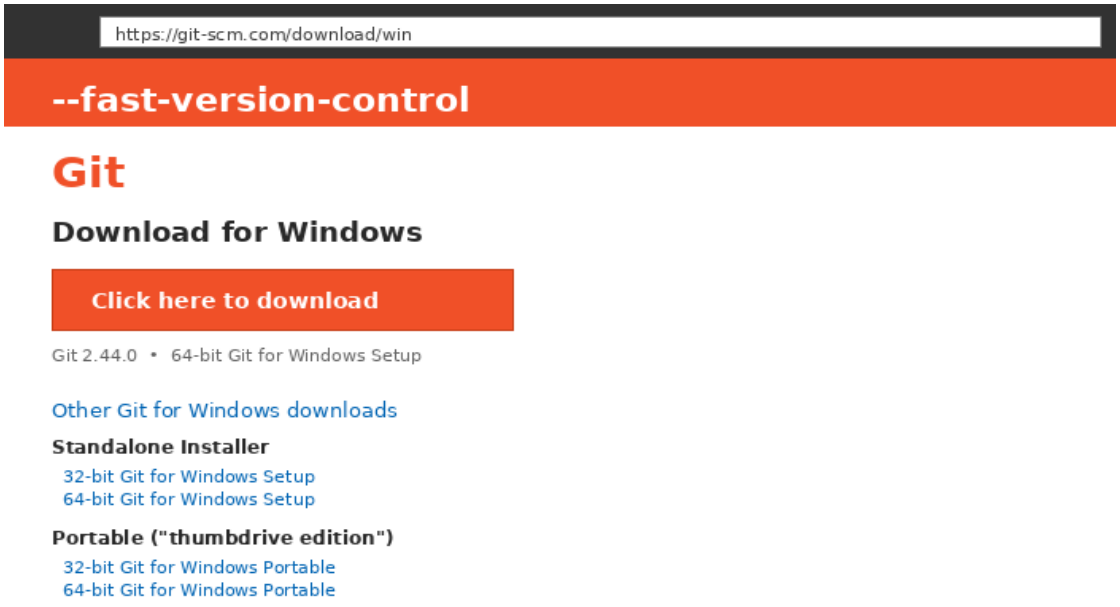

Figura 1. Página de descarga oficial de Git para Windows (git-scm.com/download/win).

4.2. Licencia GNU GPL

Al ejecutar el instalador como administrador, se presenta la pantalla con la Licencia Pública General de GNU (GPL v2). Se lee el contenido y se hace clic en Next para aceptar y continuar con la instalación.

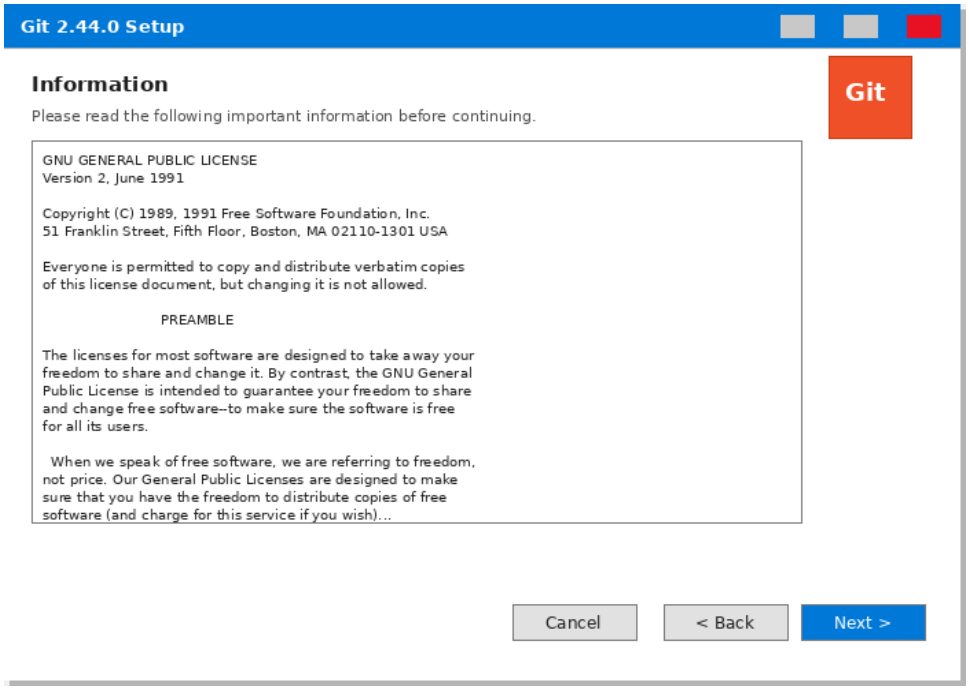

Figura 2. Pantalla de aceptación de la licencia GNU GPL del instalador de Git.

4.3. Ruta de instalación

Se mantiene la ruta predeterminada C:\Program Files\Git y se hace clic en Next. Esta es la ubicación estándar recomendada para instalaciones de sistema en Windows.

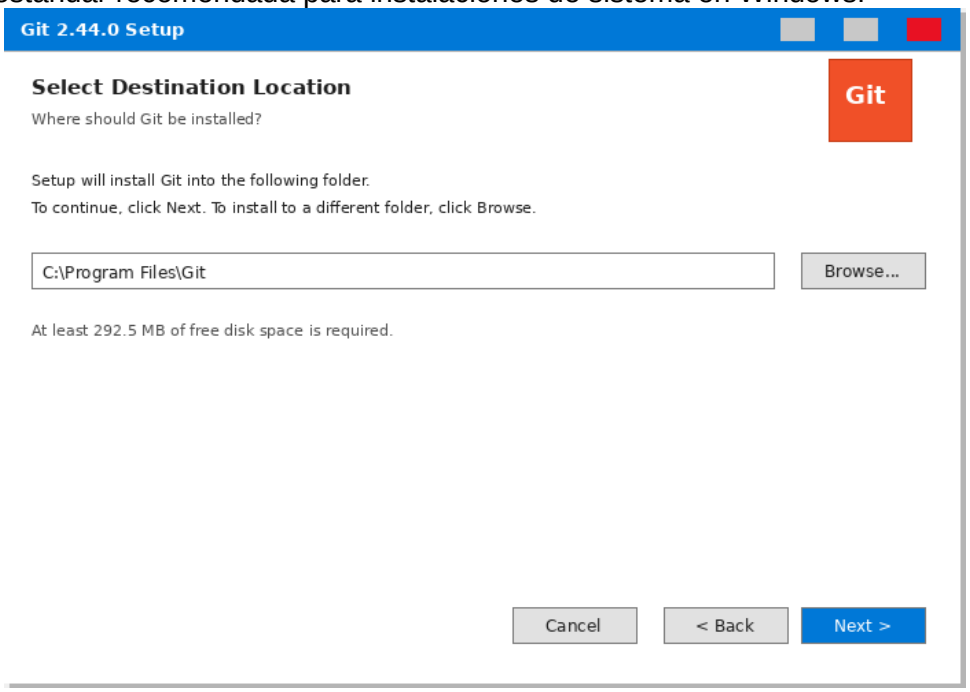

Figura 3. Selección de ruta de instalación predeterminada para Git.

4.4. Selección de componentes

Se mantienen los componentes predeterminados y se activa la opción "Add a Git Bash Profile to Windows Terminal", que permite usar Git Bash como terminal integrada en Windows Terminal.

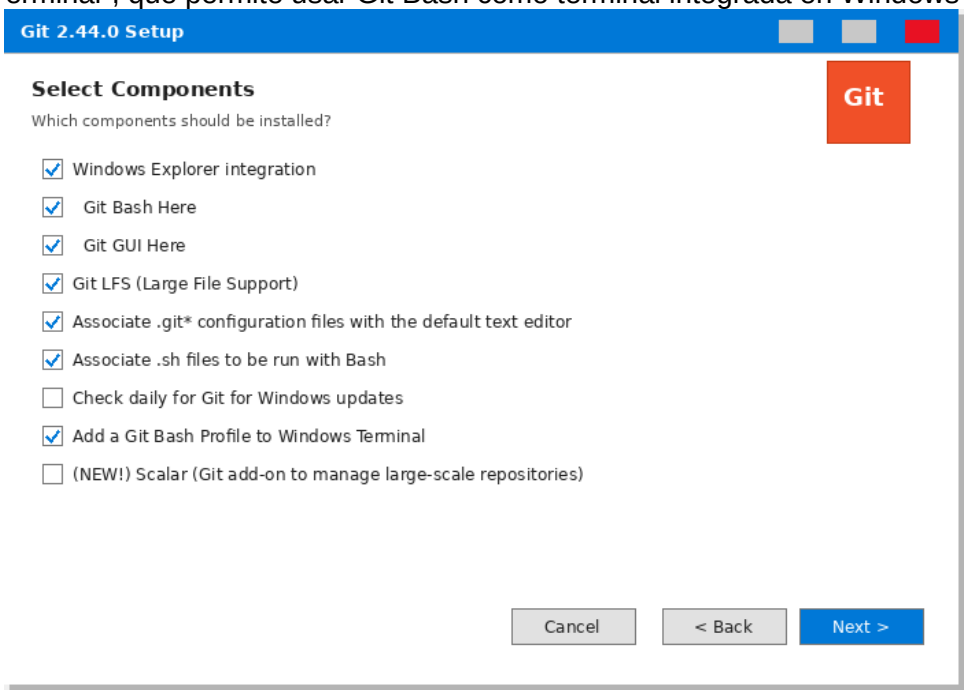

Figura 4. Selección de componentes durante la instalación de Git.

4.5. Editor predeterminado

Se selecciona Visual Studio Code como editor predeterminado de Git, dado que es el IDE principal del proyecto. Esta configuración permite que los mensajes de commit y los conflictos de fusión se abran en VS Code automáticamente.

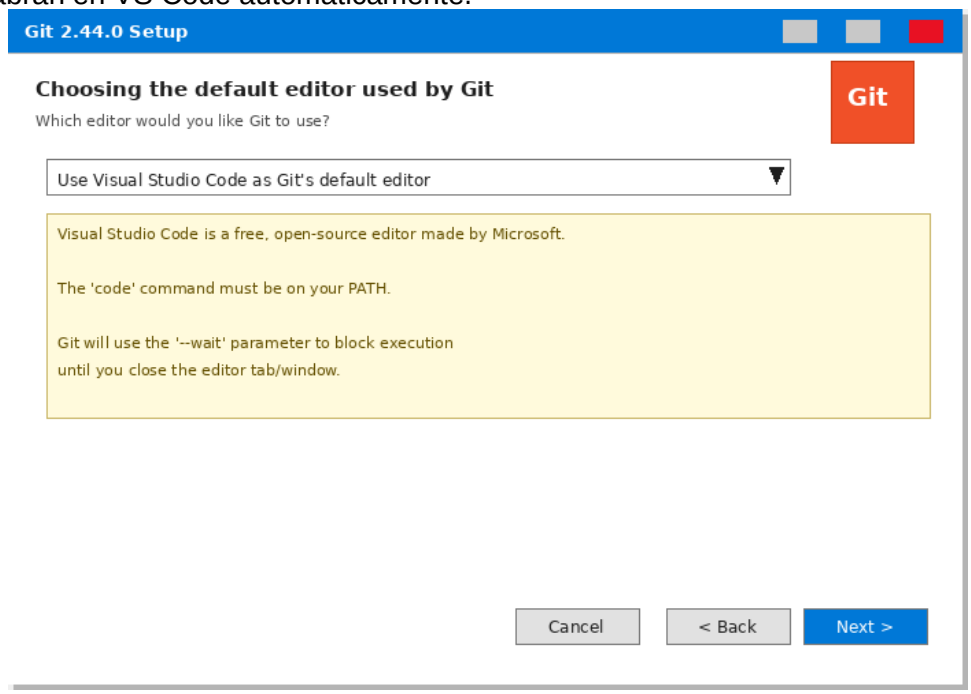

Figura 5. Configuración de Visual Studio Code como editor predeterminado de Git.

4.6. Nombre de la rama inicial

Se selecciona la opción Override the default branch name for new repositories y se escribe main como nombre de la rama inicial, en cumplimiento con la estrategia Git Flow definida en la evidencia GA7-220501096-AA1-EV01 y con la convención adoptada por GitHub.

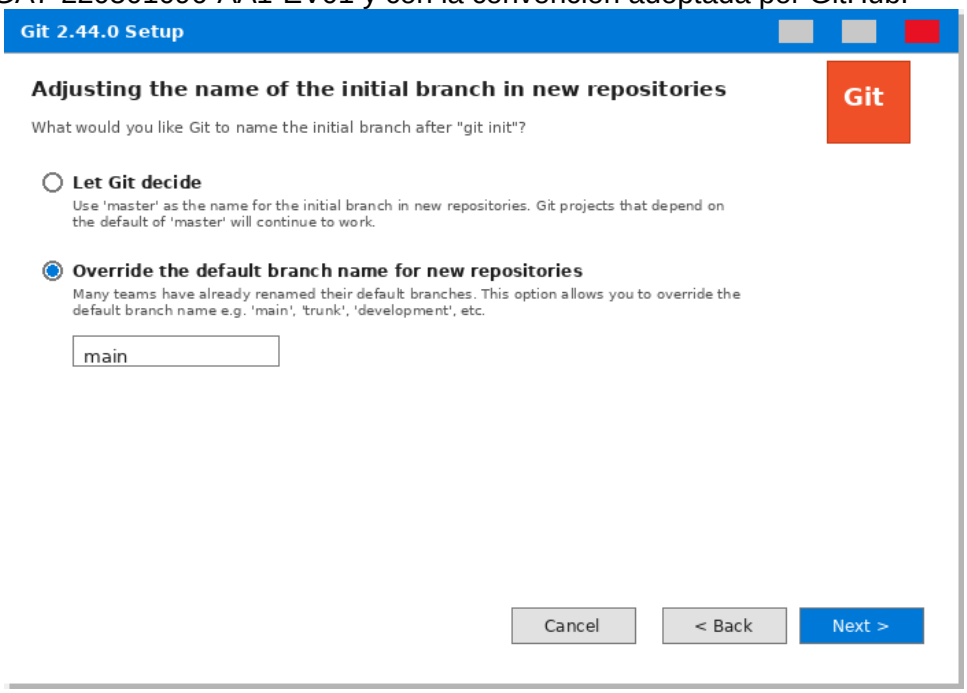

Figura 6. Configuración del nombre de la rama principal como 'main'.

4.7. Configuración del PATH del sistema

Se selecciona la opción Git from the command line and also from 3rd-party software, que agrega Git al PATH del sistema. Esto permite utilizar los comandos git desde CMD, PowerShell, Git Bash y la terminal integrada de VS Code.

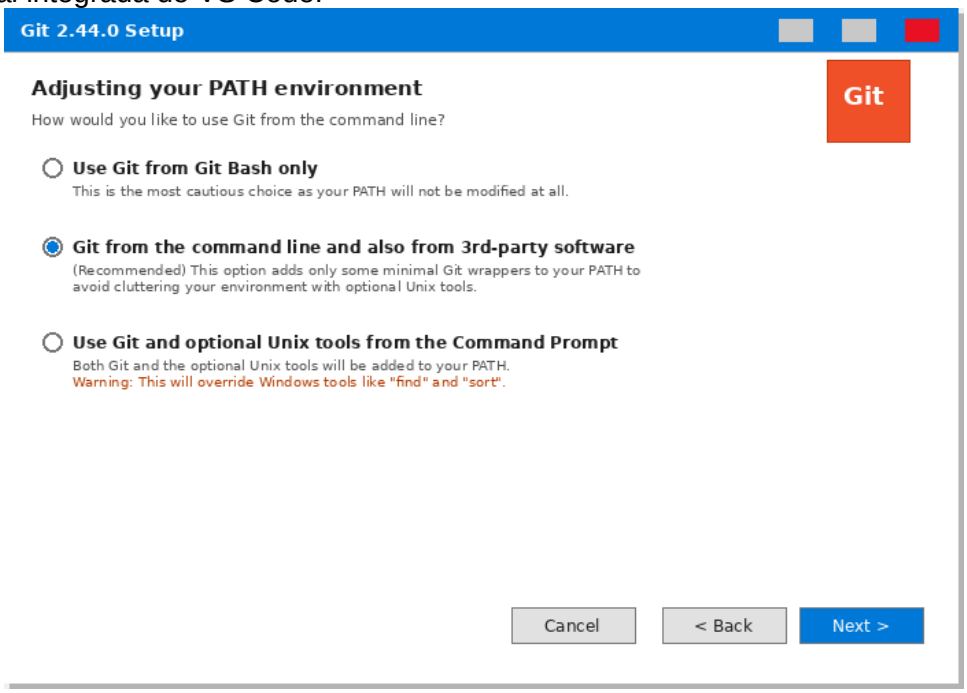

Figura 7. Configuración del PATH del sistema para acceso global a Git.

4.8. Configuración de saltos de línea

Se selecciona Checkout Windows-style, commit Unix-style line endings. Esta opción configura core.autocrlf=true, garantizando compatibilidad entre el entorno Windows del desarrollador y el pipeline CI de GitHub Actions que corre en entornos Linux.

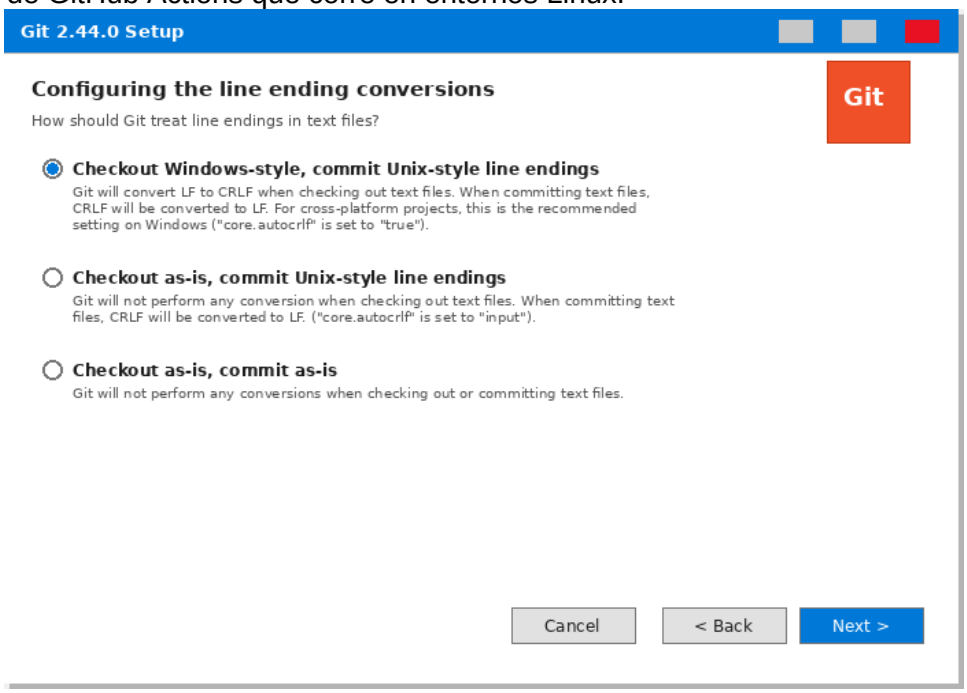

Figura 8. Configuración de manejo de saltos de línea (CRLF/LF).

4.9. Finalización e inicio de Git Bash

Se hace clic en Finish para cerrar el asistente. Se activa la opción Launch Git Bash para abrir la terminal inmediatamente y verificar la instalación.

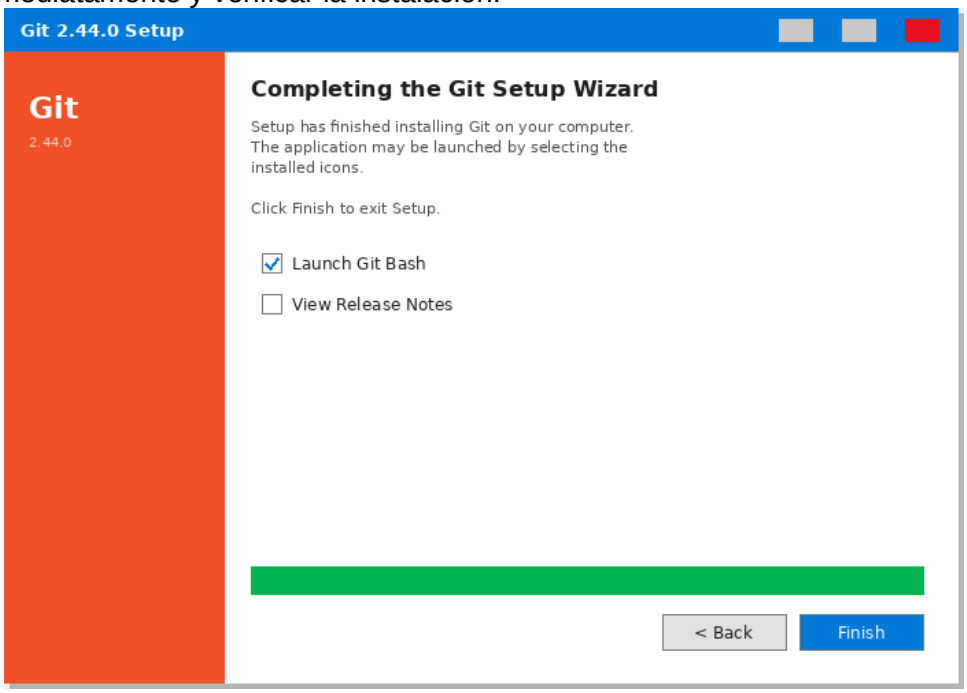

Figura 9. Pantalla de finalización del instalador de Git 2.44.0.

4.10. Verificación de la instalación

Se abre Git Bash y se ejecuta el comando git --version. La salida confirma: git version 2.44.0. windows.1, lo que indica que la instalación fue exitosa y Git está disponible globalmente en el sistema.

```
MINGW64:/c/proyectos/sistema-diseno-circuitos-v3

MINGW64 ~
$ git --version
git version 2.44.0.windows.1

MINGW64 ~
$ git config --global user.name "Michael Ronald Olivares Giraldo"

MINGW64 ~
$ git config --global user.email "michael.olivares@sena.edu.co"

MINGW64 ~
$ git config --global init.defaultBranch main

MINGW64 ~
$ _
```

Figura 10. Verificación de la versión de Git instalada mediante Git Bash.

5. CONFIGURACIÓN INICIAL DEL REPOSITORIO GIT LOCAL

Con Git correctamente instalado, se configura la identidad del desarrollador en el ámbito global. Esta configuración es obligatoria antes de realizar el primer commit, ya que Git registra el nombre y correo del autor en cada confirmación de cambios.

5.1. Configuración de identidad y parámetros globales

Se ejecutan los comandos git config --global que almacenan la configuración en el archivo ~/.gitconfig. Se establece el nombre completo, correo institucional SENA, rama predeterminada main y editor Visual Studio Code:

```
MINGW64:/c/proyectos/sistema-diseno-circuitos-v3

MINGW64 ~
$ git --version
git version 2.44.0.windows.1

MINGW64 ~
$ git config --global user.name "Michael Ronald Olivares Giraldo"

MINGW64 ~
$ git config --global user.email "michael.olivares@sena.edu.co"

MINGW64 ~
$ git config --global init.defaultBranch main

MINGW64 ~
$ _
```

Figura 11. Configuración del nombre de usuario y correo electrónico en Git global.

5.2. Verificación de la configuración global

Se ejecuta git config --global --list para visualizar todas las opciones configuradas y verificar que los valores son correctos. La salida muestra los seis parámetros establecidos:

```
MINGW64:/c/proyectos/sistema-diseno-circuitos-v3

MINGW64 ~
$ git config --global --list
user.name=Michael Ronald Olivares Giraldo
user.email=michael.olivares@sena.edu.co
init.defaultbranch=main
core.editor=code --wait
diff.tool=vscode
core.autocrlf=true

MINGW64 ~
$ _
```

Figura 12. Listado completo de la configuración global de Git verificada.

La siguiente tabla resume los parámetros de configuración establecidos:

Parámetro	Valor configurado	Ámbito
user.name	Michael Ronald Olivares Giraldo	global
user.email	michael.olivares@sena.edu.co	global
init.defaultBranch	main	global
core.editor	code --wait	global
diff.tool	vscode	global
core.autocrlf	true (Windows)	global

Tabla 2. Resumen de parámetros de configuración global de Git.

6. INICIALIZACIÓN DEL REPOSITORIO LOCAL DEL PROYECTO

Con la configuración global establecida, se procede a inicializar el repositorio local del proyecto Sistema de Diseño Integrado de Circuitos v3.0. Este proceso crea la carpeta .git con toda la estructura de objetos y referencias de Git.

6.1. Inicialización con git init y primer commit

Se navega hasta la carpeta raíz del proyecto, se ejecuta git init y, tras crear el archivo .gitignore con las exclusiones correspondientes, se agrega todos los archivos al staging con git add . y se realiza el primer commit siguiendo la convención Conventional Commits:

```
MINGW64:/c/proyectos/sistema-diseno-circuitos-v3

$ cd C:/proyectos/sistema-diseno-circuitos-v3

$ git init
Initialized empty Git repository in
C:/proyectos/sistema-diseno-circuitos-v3/.git/

$ git add .

$ git commit -m "feat(project): inicializar Sistema de
> Diseno Integrado de Circuitos v3.0"
[main (root-commit) a3f1b2c] feat(project): inicializar...
 9 files changed, 2847 insertions(+)
 create mode 100644 index.html
 create mode 100644 dashboard.html
 create mode 100644 styles.css
 create mode 100644 app.js

$ _
```

Figura 13. Inicialización del repositorio y primer commit con git init y git commit.

6.2. Verificación del historial local

Se ejecuta git log --oneline --graph --all para verificar que el historial registra correctamente el commit inicial y que la rama main apunta al commit creado con el puntero HEAD:

```
MINGW64:/c/proyectos/sistema-diseno-circuitos-v3

$ git log --oneline --graph --all
* a3f1b2c (HEAD -> main) feat(project): inicializar
|                         Sistema de Diseno Integrado v3.0

$ git status
On branch main
nothing to commit, working tree clean

$ git branch
* main

$ _
```

Figura 14. Verificación del historial de commits con git log --oneline --graph.

7. CREACIÓN DE CUENTA Y REPOSITORIO EN GITHUB

7.1. Registro de cuenta en GitHub

Se accede a github.com/signup para crear una nueva cuenta. Se completan los campos de correo electrónico institucional, contraseña y nombre de usuario michael-olivares. GitHub envía un correo de verificación para activar la cuenta.

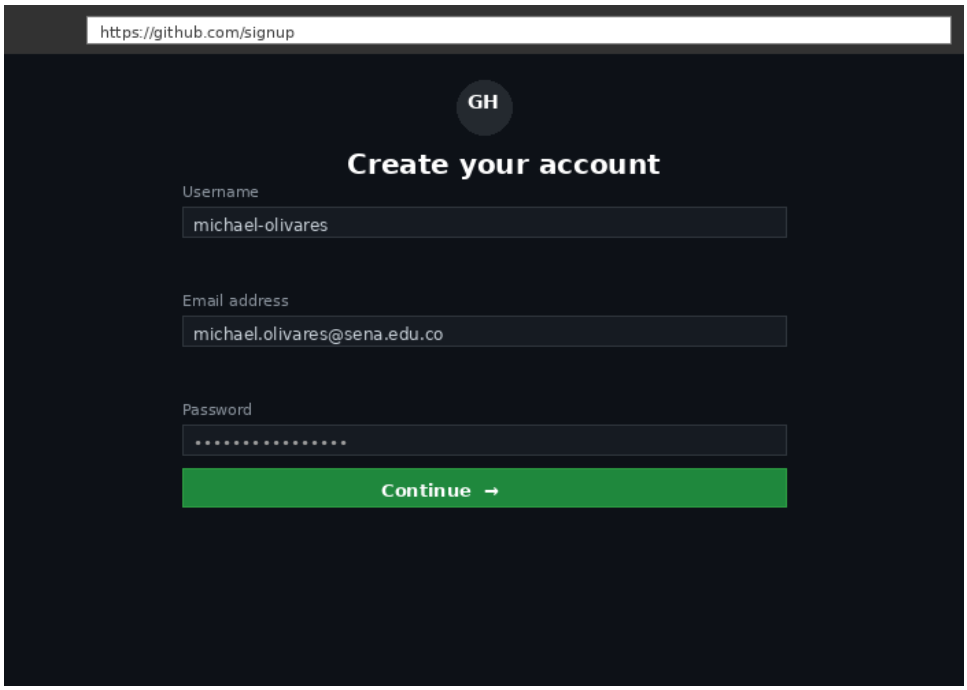

Figura 15. Formulario de registro de nueva cuenta en GitHub.

7.2. Creación del repositorio remoto

Desde el panel de GitHub se hace clic en New repository. Se configura el nombre sistema-diseno-circuitos-v3, visibilidad pública, sin inicialización (el repositorio local ya tiene commits) y licencia MIT. La URL del repositorio remoto queda como: github.com/michael-olivares/sistema-diseno-circuitos-v3.

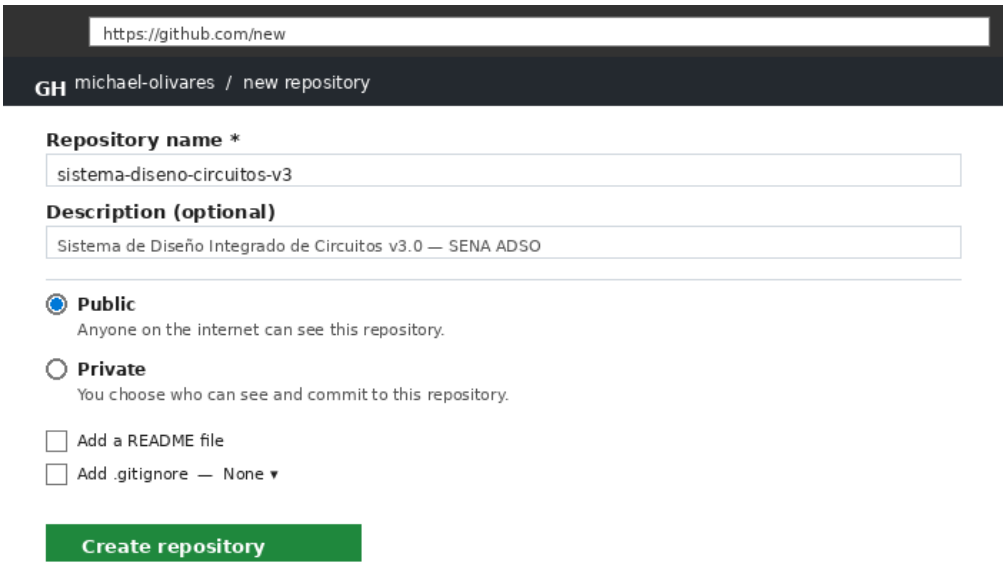

Figura 16. Formulario de creación del repositorio remoto en GitHub con los parámetros del proyecto.

Parámetro	Valor configurado
Propietario (Owner)	michael-olivares
Nombre del repositorio	sistema-diseno-circuitos-v3
Descripción	Sistema de Diseño Integrado de Circuitos v3.0 — SENA ADSO Ficha 3118306
Visibilidad	Public (repositorio público)
Inicializar con README	No (el repositorio local ya tiene commits)
Licencia	MIT License

Tabla 3. Parámetros de configuración del repositorio remoto en GitHub.

8. CONFIGURACIÓN DE AUTENTICACIÓN SSH CON GITHUB

La autenticación SSH mediante par de claves criptográficas es el método recomendado ya que elimina la necesidad de ingresar credenciales en cada push o pull, y es más seguro que HTTPS con contraseña.

8.1. Generación del par de claves SSH Ed25519

Se ejecuta ssh-keygen con el algoritmo Ed25519 (más moderno y seguro que RSA). Se especifica el correo institucional como etiqueta. Git Bash confirma la creación mostrando el fingerprint SHA256 y el diagrama randomart:

```
MINGW64:~

$ ssh-keygen -t ed25519 -C "michael.olivares@sena.edu.co"
Generating public/private ed25519 key pair.
Enter file in which to save the key
(/c/Users/michael/.ssh/id_ed25519): [Enter]
Enter passphrase (empty for no passphrase): [••••••••]
Enter same passphrase again: [••••••••]
Your identification has been saved in /c/Users/michael/.ssh/id_ed25519
Your public key has been saved in /c/Users/michael/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:xKj8mN2pQrT4uVwXyZ0aBcDeFgHiJkLmNoPqRsTuVwX michael.olivares@sena.edu.co
The key's randomart image is:
+--[ED25519 256]--+
|        . .      |
|       + o       |
|      o B .      |
|     . B S .     |
|    . . S . .    |
|   + . + . .     |
|  . = B . .      |
|   . = * . .     |
|    +oO+o.       |
+----[SHA256]-----+
```

Figura 17. Generación del par de claves SSH Ed25519 y confirmación con fingerprint.

8.2. Adición de la clave pública a GitHub

Se copia la clave pública al portapapeles con el comando: clip < ~/.ssh/id_ed25519.pub. En GitHub se navega a Settings → SSH and GPG keys → New SSH key y se pega la clave con el título SENA-ADSO-Desktop:

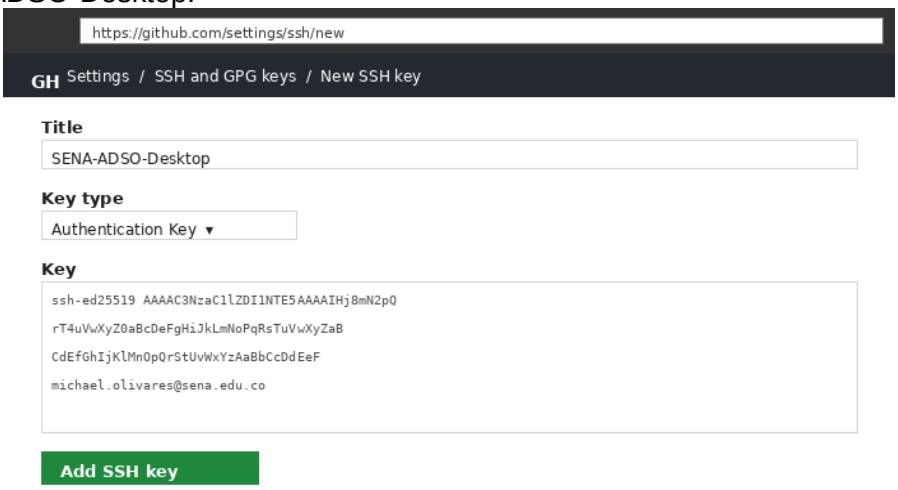

Figura 18. Adición de la clave pública SSH Ed25519 al perfil de GitHub.

8.3. Verificación y vinculación con el repositorio remoto

Se verifica la conexión SSH con ssh -T git@github.com. GitHub responde confirmando la autenticación exitosa. A continuación se registra el repositorio remoto como origin con la URL SSH y se verifica con git remote -v:

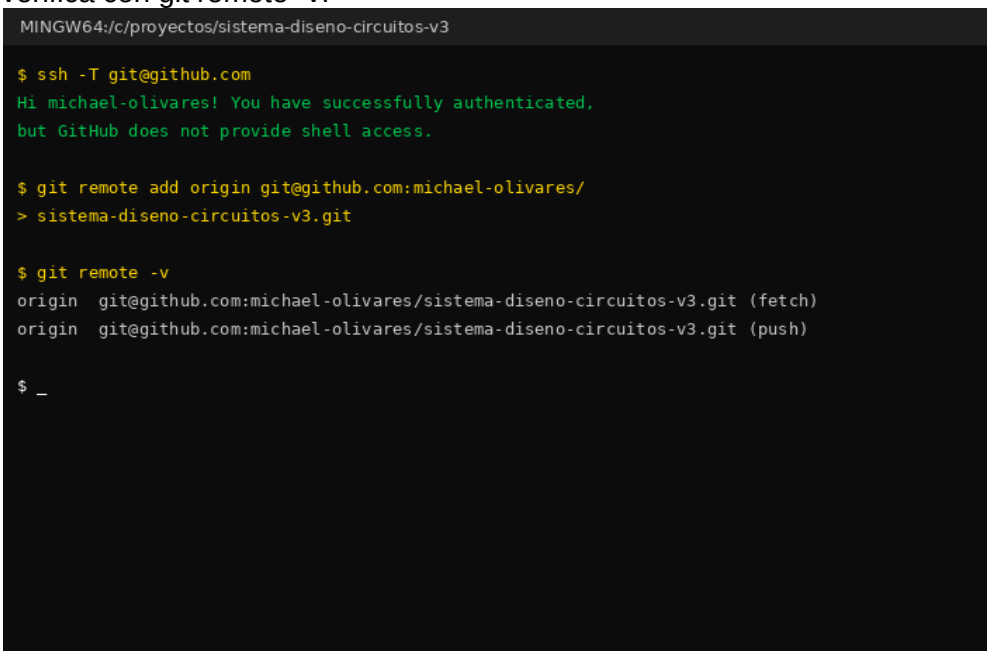

Figura 19. Verificación de la autenticación SSH y registro del repositorio remoto como origin.

9. VINCULACIÓN DEL REPOSITORIO LOCAL CON EL REMOTO Y PRIMER PUSH

Con el repositorio remoto creado y la autenticación SSH verificada, se sincroniza el historial local con GitHub mediante el primer push.

9.1. Primer push con upstream tracking

Se ejecuta git push -u origin main con el flag -u que establece el seguimiento automático entre la rama local main y origin/main. Git confirma la transferencia de objetos y el establecimiento del upstream:

```
MINGW64:/c/proyectos/sistema-diseno-circuitos-v3

$ git push -u origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (11/11), 48.32 KiB | 2.69 MiB/s, done.
Total 11 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:michael-olivares/sistema-diseno-circuitos-v3.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.

$ _
```

Figura 20. Primer push al repositorio remoto con establecimiento de seguimiento upstream.

9.2. Verificación en GitHub

Se actualiza la página del repositorio en GitHub. Se verifican los 9 archivos fuente del proyecto (7 HTML, 1 CSS, 1 JS), el archivo .gitignore, el commit inicial con mensaje Conventional Commits y la rama main activa:

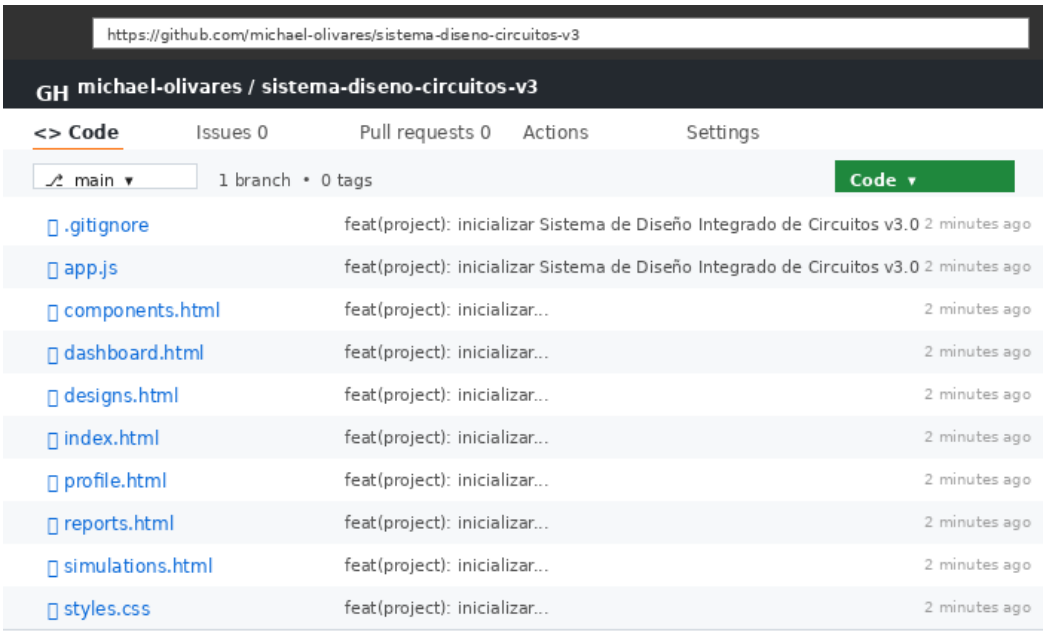

Figura 21. Repositorio remoto en GitHub con los 9 archivos del proyecto sincronizados correctamente.

10. INTEGRACIÓN DE GIT EN VISUAL STUDIO CODE

Visual Studio Code incluye integración nativa con Git. Al abrir la carpeta del proyecto, la barra lateral muestra el panel Source Control con los cambios pendientes y el indicador de rama activa. La extensión Git Graph proporciona visualización del árbol de commits:

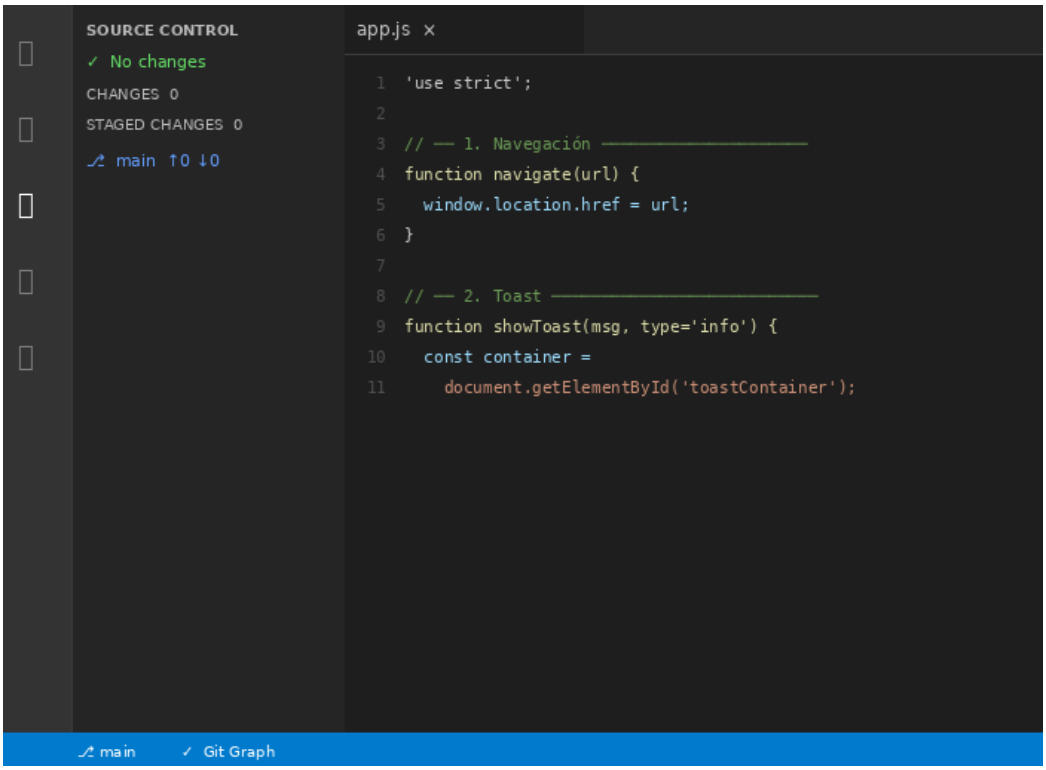

Figura 22. Panel Source Control de VS Code con el repositorio Git del proyecto y extensión Git Graph.

11. VERIFICACIÓN DEL PIPELINE DE INTEGRACIÓN CONTINUA EN GITHUB ACTIONS

Tras configurar el archivo .github/workflows/ci.yml con las reglas de validación HTML5, CSS3 y JavaScript, cada push al repositorio remoto activa automáticamente el workflow. La pestaña Actions de GitHub muestra la ejecución exitosa del pipeline CI con todos los pasos en verde:

Figura 23. Primera ejecución exitosa del pipeline CI en GitHub Actions con validaciones en verde.

12. CONCLUSIONES

- Se instaló correctamente Git 2.44.0 en el sistema operativo Windows 11, configurando Visual Studio Code como editor predeterminado, el manejo de saltos de línea CRLF/LF para compatibilidad con el pipeline CI, y la adición de Git al PATH del sistema. La verificación con git --version confirmó la instalación exitosa.

- La configuración global de Git mediante los comandos git config --global estableció la identidad del desarrollador (nombre completo y correo institucional SENA), el nombre de la rama principal como main y el editor VS Code, garantizando que todos los commits del proyecto quedarán firmados con los datos correctos del autor.

- El repositorio local del Sistema de Diseño Integrado de Circuitos v3.0 fue inicializado con git init, se creó el archivo .gitignore con las exclusiones correspondientes y se realizó el primer commit siguiendo la convención Conventional Commits definida en la evidencia GA7-220501096-AA1-EV02, estableciendo el historial base del proyecto.

- La cuenta en GitHub fue creada y el repositorio remoto michael-olivares/sistema-diseno-circuitos-v3 fue configurado como público. La autenticación SSH mediante par de claves Ed25519 fue configurada y verificada exitosamente, eliminando la necesidad de ingresar credenciales en cada operación de sincronización.

- La vinculación entre el repositorio local y el remoto se completó mediante git remote add origin y el primer push con git push -u origin main, confirmado visualmente en la interfaz web de GitHub donde se verificó la presencia de los 9 archivos fuente del proyecto y el historial de commits.

- La instalación y configuración documentada en el presente informe proporciona la infraestructura técnica completa para implementar el flujo de trabajo Git Flow definido en la evidencia GA7-220501096-AA1-EV01, garantizando que cada etapa del ciclo de vida del software cuente con el soporte de control de versiones requerido por el componente de formación Integración Continua.

13. REFERENCIAS

- Chacon, S. & Straub, B. (2023). Pro Git (3.ª ed.). Apress. https://git-scm.com/book/es/v2
- Git. (2024). Git Reference Manual. https://git-scm.com/docs
- GitHub. (2024). Generating a new SSH key and adding it to the ssh-agent. https://docs.github.com/en/authentication/connecting-to-github-with-ssh
- GitHub. (2024). Getting started with Git. https://docs.github.com/en/get-started/using-git
- GitHub. (2024). GitHub Actions Documentation. https://docs.github.com/en/actions
- GitHub. (2024). GitHub Pages Documentation. https://docs.github.com/en/pages
- Microsoft. (2024). Using Git source control in VS Code. https://code.visualstudio.com/docs/sourcecontrol/overview
- Conventional Commits. (2024). Conventional Commits v1.0.0 Specification. https://www.conventionalcommits.org/es/v1.0.0/
- SENA. (2026). Guía GA7-220501096-AA1-EV04. Tecnología en Análisis y Desarrollo de Software. Ficha 3118306.
- Olivares Giraldo, M. R. (2026). Identifica Herramientas de Versionamiento GA7-220501096-AA1-EV03. SENA Ficha 3118306.
- Olivares Giraldo, M. R. (2026). Definir Estándares de Codificación GA7-220501096-AA1-EV02. SENA Ficha 3118306.